Knowledge Fingerprint: Algebraic Epistemological Identity, Reconstructable Hashes, and Self-Healing Knowledge Bases

Michael Munz Independent Researcher

April 2026

Abstract

We present Knowledge Fingerprint (KFP), a deterministic system for characterizing the epistemological identity of knowledge units and AI agents. KFP encodes seven dimensions of knowledge — type, evidence level, stability, origin, structure, domain, and genesis — into a compact, human-readable, cryptographically verifiable fingerprint string. The system features: (1) a formally proven Meet-Semilattice composition algebra where merge operations can only degrade quality, (2) dual-fingerprinting for detecting metadata inconsistencies, (3) agent behavioral profiling with drift detection, (4) Reconstructable Knowledge Hashes (RKH) — a three-layer format that encodes epistemological dimensions, semantic topic keys, and content hashes into a single string from which an LLM can regenerate the original knowledge, (5) a quality-gated Wiki layer with Knowledge DNA — a compact, human-readable format in which a complete knowledge base can be stored as one line per article and self-healed through regeneration, and (6) epistemological RAG filtering — a pre-retrieval safety layer that prevents speculative or low-evidence chunks from entering LLM context. KFP is implemented in standard-library Python (33 modules, 914 tests, 8,875 lines of code, 7 MCP tools, 4 integration bridges + 1 specification), installable via `pip install kfp-fingerprint`, requires no LLM for evaluation, and is the first system to combine epistemological typing, composition algebra, cryptographic verification, semantic reconstruction, and evidence-gated

retrieval in a single framework. In validation: dual-fingerprinting detected 82% domain misclassification, template-based reconstruction achieves 76% mean dimension fidelity without LLM, 191 real-world articles were ingested as ~19 KB Knowledge DNA and regenerated in under 0.4 seconds, epistemological RAG filtering reduced unsafe chunk leakage by 86% against a 500-annotation ground truth benchmark, and a PubMedQA-inspired medical evidence benchmark (50 sentences) achieved 66% accuracy rule-based and 82% with LLM-assisted classification across 6 evidence levels.

1. Introduction

Knowledge management systems, retrieval-augmented generation (RAG) pipelines, and multi-agent frameworks share a fundamental gap: they treat all knowledge identically. A peer-reviewed clinical trial result, a blog opinion, and a market prediction are stored as vectors of the same shape, retrieved by the same similarity function, and merged without regard to their epistemological nature.

This creates four problems:

1. **Silent quality degradation.** When knowledge from different evidence levels is merged, the quality information is lost. A system cannot distinguish “empirical + speculative = mixed” from “empirical + empirical = strong.”
2. **Metadata-content divergence.** Metadata systems (tiers, quality flags, confidence scores) can become stale or incorrect. Without comparing metadata against actual content, misclassification goes undetected.
3. **Agent role inconsistency.** In multi-agent systems, agents are assigned roles (auditor, researcher, strategist) but their actual output may not match. No existing tool measures whether an agent produces knowledge consistent with its declared role.
4. **Unsafe retrieval.** RAG pipelines retrieve by semantic similarity without considering evidence quality. A speculative blog post about drug interactions ranks equally

with a peer-reviewed clinical trial — and both enter the LLM's context window.

KFP addresses all four problems with a unified framework: a 7-dimensional taxonomy that describes what *kind* of knowledge something is, a closed composition algebra that makes quality degradation explicit and irreversible, an agent identity system that measures role consistency, drift, and team diversity, and an evidence-gated retrieval layer that filters unsafe knowledge before it reaches the LLM.

1.1 End-to-End Example

Figure 1: KFP pipeline — from input text through fingerprint extraction, risk scoring, reconstructable hashing, Knowledge DNA, RAG filtering, to merge guard.

The following walkthrough traces a single sentence through the entire KFP pipeline.

Input:

```
"A randomized controlled trial (n=5000) demonstrated that
aspirin 100mg
daily reduces cardiovascular events by 25% compared to placebo
(p<0.001)."
```

Step 1 — Fingerprint extraction. KFP's rule-based extractor analyzes the text and produces a 7-dimensional fingerprint:

```
v2:causal:empirical:slow_decay:primary:claim:medical:learned@a7f
3b2c9e1d4
```

Each dimension is readable: this is *causal* knowledge (aspirin reduces events), *empirically* supported (RCT), with *slow decay* stability (medical evidence evolves over years), from a *primary* source, structured as a *claim*, in the *medical* domain.

Step 2 — Risk score. The quality gate computes a risk score: **0.05** (safe). Evidence is empirical, stability is slow_decay, extraction confidence is high. Well below the 0.4 flag threshold.

Step 3 — RKH (Reconstructable Knowledge Hash). The fingerprint is extended with a semantic topic key:

```
rkh1:causal:empirical:slow_decay:primary:claim:medical:learned
     :aspirin.cardiovascular_reduction.25_percent@a7f3b2c9e1d4
```

Three layers in one line: epistemological (what kind), semantic (which knowledge), cryptographic (is it the original).

Step 4 — Reconstruction. From the RKH alone (without the original text), a template-based reconstructor generates:

```
"Studies show that aspirin causes cardiovascular reduction
(25 percent). [empirical, slow_decay]"
```

Not bit-identical, but epistemologically faithful — the dimensions match. An LLM-assisted reconstructor produces higher-fidelity text.

Step 5 — Knowledge DNA. In a wiki of 191 articles, this sentence becomes one line:

```
rkh1:causal:empirical:...:aspirin.cardiovascular_reduction@a7f3
# Aspirin RCT
```

The entire wiki fits in ~19 KB. If the original articles are lost, the wiki regenerates itself from these lines.

Step 6 — RAG filtering. When this chunk enters a RAG pipeline alongside a speculative blog post ("My neighbor says aspirin cured his headache"), KFP's `safe_retrieve()` passes the RCT (empirical) and blocks the anecdote (anecdotal) — before the LLM ever sees it.

Step 7 — Merge guard. If someone tries to merge this empirical finding with a speculative claim, the algebra warns: "dangerous merge — evidence gap too large (empirical to speculative = 4 ordinal steps)." The merge would produce a `speculative` result, making the quality loss explicit.

1.2 Design Principles

1. **Deterministic First.** All modules function without an LLM. Rule-based extraction (300+ regex patterns, English + German + code) is the primary path, with a Naive Bayes fallback classifier for cases where patterns do not match. LLMs may accelerate extraction but are never required.
2. **Compact over Complete.** A fingerprint fits in one line: `v2:causal:empirical:slow_decay:primary:claim:medical:innate@a7f3b2c9e1d4`. Seven dimensions plus a hash suffix.
3. **Human-Readable.** Every dimension value is a plain-text English word. No lookup tables needed.
4. **Composable.** The algebra is closed: any operation on fingerprints produces valid fingerprints.
5. **Verifiable.** A SHA-256 hash suffix proves content integrity without revealing content.
6. **Zero Dependencies.** The core framework runs on Python's standard library alone. LLM providers, vector stores, and graph databases are optional bridges.

2. The Fingerprint Model

2.1 Seven Dimensions

Dimension	Values	Ordinal?	Role
Knowledge Type	causal, correlative, definitional, heuristic, normative, predictive, procedural	Partial (breadth)	What kind of knowledge
Evidence	empirical, consensus, theoretical, anecdotal, speculative, contested	Total	How well supported
Stability	immutable, slow_decay,	Total	How long it remains valid

Dimension	Values	Ordinal?	Role
	fast_decay, volatile		
Origin	primary, derived, aggregated, synthesized	None	Where it came from
Structure	taxonomy, claim, argument, narrative, dataset	None	What form it takes
Domain	medical, technical, legal, physics, general	None	What field it belongs to
Genesis	innate, learned, transmitted, emergent	None	How it was created

The total combinatorial space is 7 x 6 x 4 x 4 x 5 x 5 x 4 = **67,200 semantic combinations** (16,800 excluding genesis). Each combination represents a distinct epistemological identity.

2.2 Extraction

KFP extracts fingerprints from text using a three-tier classification pipeline:

1. **Rule-based (primary).** 300+ regex patterns covering English, German, and source code. Domain is auto-detected from vocabulary. Knowledge type is inferred from linguistic markers (causal: “causes”, “leads to”, “inhibits”; procedural: “step 1”, “to do this”; normative: “should”, “must”, “guideline”).
2. **Naive Bayes fallback.** When regex patterns produce no confident match, a stdlib-only Naive Bayes classifier (word unigrams + bigrams) provides `predict_evidence()` and `predict_knowledge_type()`. This eliminates the hard fallback to default values that plagued earlier versions.
3. **LLM-assisted (optional).** A 7-dimension LLM classifier with thread-safe in-process caching (keyed by content hash) classifies all dimensions in a single call. Cache

statistics (hits/misses/size) enable monitoring. Batch API (`extract_batch`) supports bulk classification. No LLM is required — this tier is strictly opt-in.

The rule-based and Naive Bayes tiers are fully deterministic: the same text always produces the same fingerprint.

2.3 Risk Score

Every fingerprint carries a **risk score** (0.0 = safe, 1.0 = high risk) computed from evidence level, stability, and confidence. The quality gate uses configurable thresholds:

- **< 0.4**: Pass (safe for use)
- **0.4 - 0.8**: Flag for review
- **> 0.8**: Block (reject from pipeline)

Risk factors are returned as structured explanations ("evidence is speculative", "stability is volatile", "low extraction confidence"). The `safe_retrieve()` RAG wrapper uses the risk score as its primary filter.

2.4 Serialization

```
v2:causal:empirical:slow_decay:primary:claim:medical:innate@a7f3b2c9e1d4
```

The format is versioned (`v2:` prefix), grep-able, diff-able, and pipe-friendly. The hash suffix (SHA-256, first 12 hex characters) enables integrity verification without content access.

3. The Composition Algebra

3.1 Meet-Semilattice Structure

The merge operation `merge(F1, F2)` is defined dimension-wise:

- **Evidence, Stability:** Take the minimum (weaker value wins)
- **Knowledge Type:** Take the maximum breadth
- **Origin:** Constant "synthesized" (absorbing element)
- **Structure, Domain:** Keep if equal, else fall back to "argument"/"general"

- **Hash:** Recompute from combined content

3.2 Proven Properties

Property	Status	Implication
Commutativity	Proven + tested	`merge(A, B) == merge(B, A)`
Associativity	Proven + tested	Grouping doesn't matter
Monotonicity	Proven on ordinal dims	Quality can only degrade
Absorption	6 elements identified	Once contested/volatile, always contested/volatile
Closure	Proven + tested	Any operation produces valid fingerprints
Non-idempotence	Proven	`merge(F, F) != F` (origin absorbs)

Key insight: `merge` is destructive. One weak source drags everything down irreversibly. This is not a bug — it is the core feature. It makes quality degradation *visible before it happens*.

The `derive` operation preserves quality: derivation does not degrade evidence or stability.

Formal proof: see `concepts/algebra_proof.md`.

3.3 Practical Consequence: Merge Guard

Before merging two knowledge units, the algebra predicts the resulting quality. If the merge would cause evidence to drop from `empirical` to `speculative` (4 ordinal steps), the system warns: "dangerous merge — evidence gap too large."

4. Dual-Fingerprinting: Inconsistency Detection

4.1 Method

For each knowledge unit, KFP creates two independent fingerprints:

1. **Adapter fingerprint:** Derived from structured metadata (tier, quality flag, confidence score)
2. **Extractor fingerprint:** Derived from actual text content (regex-based analysis)

Divergence between the two fingerprints indicates that the metadata does not accurately describe the content.

4.2 Results

Scanning 39 knowledge base chunks with dual-fingerprinting revealed:

- **82% domain misclassification:** Content tagged as `medical` was actually `technical` (market projections, software specifications)
- **100% had at least one metadata inconsistency**
- The existing quality system (immune-system-style threat detection) missed all of these

This finding suggests that metadata-content divergence is a pervasive problem in knowledge management systems, and that dual-fingerprinting is an effective detection mechanism.

4.3 Limitations

The 82% figure comes from a single internal dataset (39 chunks). Independent replication on public datasets is needed to establish generalizability.

5. Agent Identity

5.1 Insight

If you can fingerprint knowledge, you can fingerprint knowledge-producers. The same 7-dimensional algebra works identically on agent output portfolios. No changes to the algebra were needed.

5.2 Two Orthogonal Components

Knowledge Identity = the distribution of an agent's outputs across 7 KFP dimensions. An agent producing 80% heuristic + 20% speculative has a different identity than one producing 60% causal + 40% empirical.

Behavioral Identity = how the agent works. Five dimensions: decisiveness, tool diversity (Shannon entropy), read/write ratio, autonomy, average sequence length. Extracted from tool traces.

5.3 YAML Role Definitions

Agent roles are defined declaratively in `role_expectations.yaml`, specifying per-role constraints:

- **expected_types**: Which knowledge types the role should produce
- **min_evidence**: Minimum evidence level for outputs
- **expected_behavior**: Behavioral constraints (decisiveness, tool diversity)

Six built-in roles are provided (auditor, librarian, strategist, red_team, researcher, executor). Custom roles can be added without code changes. The YAML file ships with the `kfp-fingerprint` package.

5.4 Role Consistency

KFP's inconsistency detector, originally designed for metadata-content divergence, applies directly to agents: compare the **declared role** (from YAML definition) against the **actual output portfolio**.

First live result: a "Strategist" agent was diagnosed as "critical" — 67% of its outputs were speculative, with zero predictive

claims. Its declared role implied strategic foresight; its actual output was speculation.

5.5 Team Blind-Spot Analysis

`compare_team()` analyzes N agent profiles simultaneously:

- **Diversity Score:** What fraction of 7 knowledge types does the team cover?
- **Blind Spots:** Which types/domains does *no* agent produce?
- **Overlap:** Which agents produce identical output profiles?

First live result (4-agent debate): - Diversity: 0.29 (2 of 7 types covered) - Blind spots: causal, correlative, normative, predictive, procedural - 3 agents had 100% identical knowledge fingerprints

These findings were invisible to the existing consensus mechanism.

5.6 Drift Detection

KFP tracks how an agent's identity evolves across sessions:

- **Shift** (A to B): Expected during task changes
- **Gradual** (A to B to C): Consistent drift, may indicate learning or degradation
- **Oscillation** (A to B to A): Unstable identity — concerning

Oscillation triggers "major" severity because it indicates the agent lacks a stable identity.

6. Knowledge Units

A Knowledge Unit bundles a KFP fingerprint, a SHA-256 provenance chain, and a version history into a single portable, versioned, cryptographically verifiable container.

6.1 Operations

- `create(content, domain, unit_id)` — new Unit with initial fingerprint
- `update(unit, new_content)` — new Unit with diff and provenance step
- `verify(unit)` — checks provenance chain integrity

- `to_json / from_json` — lossless serialization

6.2 Design Rationale

Existing knowledge containers (vector store chunks, MCP transcripts, wiki pages) lack one or more of: identity (what kind of knowledge), genealogy (how it evolved), and verifiability (proof of integrity). The Unit provides all three atomically.

7. Reconstructable Knowledge Hash (RKH)

7.1 Motivation

A KFP fingerprint answers “what kind of knowledge is this?” but not “which knowledge specifically?” The SHA-256 hash suffix enables verification but not reconstruction. If the original text is lost, the fingerprint becomes an empty identity card.

The Reconstructable Knowledge Hash (RKH) extends the fingerprint with a **semantic topic key** — a hierarchical address compact enough for one line but rich enough for an LLM to regenerate the knowledge.

7.2 Three-Layer Format

```
rkh1:causal:empirical:slow_decay:primary:claim:medical:innate
     :aspirin.cox_inhibition.irreversible@a7f3b2c9e1d4
```

Layer	Fields	Function	Decodable?
Epistemological	7 KFP dimensions	What KIND of knowledge	Directly readable
Semantic	Topic key (entity.aspect.qualifier)	WHICH knowledge	LLM-reconstructable
Cryptographic	SHA-256 (12 hex chars)	IS IT the original?	Verification only

7.3 Topic Key Extraction

Topic keys follow a hierarchical dot-separated format: `entity.aspect.qualifier` (maximum 3 levels). Extraction is deterministic:

1. **Domain-specific entity recognition.** Medical entities from a canonical code database (59 drugs, 32 lab values, 13 conditions with ATC/LOINC/SNOMED codes). Legal, physics, and technical entities from configurable dictionaries (`topic_entities.json`). Confidence: 0.8-1.0.
2. **Noun phrase extraction.** Capitalized phrases, ALL-CAPS terms, hyphenated compounds. Confidence: 0.5.
3. **Relation markers.** 25+ patterns (English + German) plus domain-specific markers. Maps verbs to canonical forms: “inhibits” to `inhibition`, “reduces” to `reduction`.
4. **Qualifier extraction.** Numbers with units, named qualifiers (irreversible, chronic, acute).

Each extraction carries a confidence score (1.0 = dictionary match, 0.8 = domain entity, 0.5 = noun phrase, 0.2 = word fallback) that propagates through the entire pipeline.

7.4 Reconstruction

From an RKH, the original knowledge can be reconstructed at two fidelity levels:

Template-based (no LLM required): Uses the 7 dimensions + topic key to generate a structured statement. Evidence prefix (“Studies show that…”), knowledge type template (“X causes Y”), stability suffix (“as of current data”). Benchmark result: **76% mean dimension fidelity** (90% per-dimension match rate, 33% full-dimension match) across 15 diverse inputs spanning 5 domains.

LLM-assisted: A structured prompt encodes all 7 dimensions + topic key as constraints. Any LLM callable can serve as backend (Claude, GPT, Ollama). The reconstruction is verified by re-extracting a fingerprint and comparing dimensions.

Key property: Reconstruction is **lossy but faithful**. The text is not bit-identical to the original (hash will not match), but the epistemological identity is preserved (dimensions should match).

This is analogous to JPEG vs. PNG — semantic fidelity, not bit fidelity.

7.5 Prior Art Assessment

Seven systems were evaluated against RKH's three-layer combination:

System	Epistemological	Semantic Key	Crypto Hash	Regenerable	Readable
Semantic Hashing (Salakhutdinov 2009)	No	No	No	No	No
IPFS CID	No	No	Yes	No	Partial
MyMories .mmr	Partial	No	Checksum	Partial	No
Knowledge Card (ICLR 2024)	No	No	No	Yes (full model)	No
Dublin Core / PROV-O / SKOS	Partial	Yes (SKOS)	No	No	Partial
Semantics-Reconstructing Hashing	No	No	No	No	No
Caveman Compression	No	No	No	Partial	Partial

No existing system combines all three layers into a single compact, human-readable identifier from which an LLM can reconstruct the knowledge semantically.

8. LLM-Wiki and Knowledge DNA

8.1 Architecture

The Wiki layer builds on KFP fingerprints, RKH, and Knowledge Units to create a quality-gated knowledge base:

- Every article is a `KnowledgeCapsule` (versioned, cryptographically verifiable)
- Every article carries an RKH (reconstructable semantic address)

- Quality gates enforce minimum evidence levels at creation time
- Cross-references are detected automatically via topic key overlap
- Consistency checks find contradictions, blind spots, and evidence warnings

8.2 Quality Gates

Articles can be gated on evidence level at creation:

```
wiki.create_article(content, title="...", min_evidence="empirica
        l")
# raises QualityGateError if evidence is below threshold
```

The consistency checker reports: - **Contradictions:** Same domain + same knowledge type + contested evidence - **Blind spots:** Knowledge types not covered by any article - **Evidence warnings:** “57% speculative”, “no empirical articles” - **Topic confidence warnings:** “35% of articles have unreliable topic keys”

8.3 Knowledge DNA

The most compact representation of a wiki is its **Knowledge DNA** — one RKH line per article:

```
# Knowledge DNA — 3 articles
rkh1:causal:empirical:...:aspirin.cox_inhibition@a7f3  # Aspirin
COX
rkh1:normative:consensus:...:dsgvo.right_to@b409      # GDPR
Art. 17
rkh1:definitional:consensus:...:sha256.hashing@9a22    # SHA-256
```

Properties: - **13x more compact** than JSON serialization (validated on 3-article wiki) - **Human-readable:** Domain, evidence, topic visible in plain text - **Git-diffable:** One line per article, sorted, comments for titles - **Self-healing:** A wiki can regenerate itself from its DNA

8.4 Validation: 191-Article Ingest

To validate the full pipeline, we ingested 191 real-world articles from diverse sources (social media captures, research notes, tool evaluations):

Metric	Value
Articles ingested	191/191 (0 failures)
DNA file size	~19 KB
Regeneration time	~0.3 seconds
Topic confidence 1.0 (dictionary match)	34 articles (18%)
Topic confidence 0.8 (domain entities)	90 articles (47%)
Topic confidence 0.2 (word fallback)	67 articles (35%)
Blind spots	correlative, procedural
Evidence warning	57% speculative

The confidence distribution reveals that 65% of articles have reliable topic keys (confidence >= 0.8), while 35% fall back to generic extraction. This aligns with the domain coverage: articles about known medical drugs or technical systems get high confidence; general articles about trends or opinions do not.

8.5 DNA as Exchange Format

Knowledge DNA has properties analogous to a Software Bill of Materials (SBOM) for code:

- **SBOM answers:** “What software components are in this system?”
- **Knowledge DNA answers:** “What knowledge is in this system, at what evidence level?”

Two agents can compare their DNA to find: shared knowledge, contradictions, complementary coverage, and blind spots — without exchanging the full text.

9. Epistemological RAG Filtering

9.1 Motivation

Standard RAG pipelines retrieve by semantic similarity: the chunks most similar to the query enter the LLM's context window. But similarity says nothing about reliability. A speculative blog post and a randomized controlled trial may both discuss "aspirin and cardiovascular risk" — and both score equally in cosine similarity.

This is dangerous. If speculative chunks enter the context alongside empirical ones, the LLM has no way to distinguish them. The generated answer silently mixes evidence levels, presenting speculation as if it were established fact.

9.2 Method

KFP adds an epistemological pre-filter to the retrieval pipeline:

```
from kfp.rag import safe_retrieve

# One line makes RAG safer
results = safe_retrieve(chunks, query, min_evidence="consensus")
```

The filter works in three steps:

1. **Fingerprint all chunks** — each chunk gets a KFP fingerprint with evidence level
2. **Filter by evidence threshold** — only chunks at or above `min_evidence` pass
3. **Return safe results** — with portfolio analysis (evidence distribution, domain coverage, blind spots)

This is a **pre-retrieval** filter. It operates before the LLM sees any content. It is complementary to post-generation evaluation tools like RAGAS, which assess output quality after the LLM has already been exposed to the chunks.

9.3 Ground Truth Benchmark

To validate the filter, we created two benchmark datasets:

Ground Truth v3 — a curated benchmark dataset: - **500 annotations** across 5 domains (medical, technical, legal,

physics, general) - **Expert-labeled** with expected KFP fingerprint dimensions (AI-generated sentences, author-reviewed labels)

PubMedQA-inspired benchmark — 50 realistic medical sentences: - Covers all 6 evidence levels (consensus: 20, empirical: 13, theoretical: 7, contested: 5, anecdotal: 3, speculative: 2) - Includes deliberately tricky cases: numeric consensus claims, unmarked empirical statements, multi-clause sentences mixing evidence levels - Designed to stress-test evidence classification in the medical domain - **Results:** 66% accuracy (rule-based), 82% accuracy (LLM-assisted classification)

Results on medical RAG simulation (22 chunks, mixed evidence levels):

Metric	Vanilla RAG	KFP-Enhanced
Total chunks	22	12
Unsafe chunks (speculative/anecdotal)	7	**1**
Unsafe chunk reduction	—	**86%**

KFP filtering reduced unsafe chunk leakage by 86%, removing 6 of 7 speculative and anecdotal chunks from the retrieval results. The single remaining leak is a borderline case where rule-based extraction classified the evidence level above the threshold. The 45% reduction in total chunk count represents substantial noise removal.

9.4 The Pre-Retrieval / Post-Generation Stack

KFP and existing RAG evaluation tools (RAGAS, DeepEval) are complementary, not competing:

Layer	Tool	When	What it checks
Pre-retrieval	**KFP**	Before LLM sees chunks	Evidence quality of input
Post-generation	RAGAS	After LLM produces output	Answer faithfulness, relevance

Together they form a complete evidence quality stack: KFP prevents garbage from entering the context, RAGAS catches hallucinations in the output. Neither alone is sufficient.

9.5 Limitations

The benchmark uses a simulated RAG pipeline (no vector store, no embedding model). Results on production RAG systems with real embedding-based retrieval may differ. The 86% reduction (not 100%) demonstrates that rule-based filtering has limits — the remaining 14% leakage occurs on borderline cases where evidence classification is ambiguous.

10. Integration Bridges

KFP is designed as a composable primitive that integrates into existing systems through lightweight bridges:

10.1 TRE Bridge

The Trustless Reasoning Engine (TRE) uses multi-model debate for adversarial verification. The KFP bridge provides epistemological analysis of debate outputs — detecting when debaters produce identical knowledge fingerprints (echo chamber) or when the team has blind spots in specific knowledge types.

10.2 LangChain Adapter

A drop-in adapter for LangChain RAG pipelines:

- `fingerprint_documents(docs)` — adds KFP metadata to LangChain Documents
- `filter_by_evidence(docs, min_level)` — evidence-gated retrieval
- `safe_retrieve(docs, query, min_evidence)` — one-line safe RAG
- `retrieval_portfolio(docs)` — portfolio analysis of retrieval results

10.3 Neo4j Adapter

Maps KFP fingerprints to a graph database schema. Knowledge units become nodes with dimension properties; relationships encode provenance chains and composition operations. Enables

graph queries like “find all empirical knowledge in the medical domain that has been merged with speculative sources.”

10.4 Capture Wiki Bridge

Connects a general-purpose content capture system to the KFP Wiki. Captured ideas, articles, and bookmarks are automatically fingerprinted, assigned RKH addresses, and ingested into the wiki with quality gates. This bridge powered the 191-article validation.

10.5 Letta Memory Specification

A specification (not yet implemented) for mapping KFP Knowledge Units to the Letta (formerly MemGPT) agent memory architecture. Defines how epistemological metadata maps to Letta’s core memory, archival memory, and recall memory tiers.

11. Related Work

11.1 Agent Evaluation Tools

The agent evaluation market (2024-2026) is crowded on the **quality/observability** axis:

Tool	Core Approach	Agent Identity?	Algebra?	RAG Safety?
LangSmith	Trace + evaluation	No	No	No
AgentOps	Session recording	No	No	No
RAGAS	RAG metrics	No	No	Post-generation
Braintrust	Experiment management	No	No	No
DeepEval	Metric library	No	No	Post-generation
KFP	**Epistemological algebra**	**Yes**	**Yes**	**Pre-retrieval**

No existing tool answers “what kind of knowledge does this agent produce?” All answer “how good is the output?” KFP is complementary, not competing.

11.2 Knowledge Typing

- **GRADE Framework** (clinical evidence): Single-dimension evidence grading. Inspiration for KFP's evidence dimension, but no composition algebra.
- **Dublin Core / Schema.org:** Describe documents (containers), not knowledge (content).
- **PROV-O (W3C):** Describes provenance processes, not epistemological nature.
- **Epistemic Logic:** Formal modal logic for knowledge/belief. Theoretical foundation but no implementation-oriented algebra.

11.3 Content Hashing and Reconstruction

- **Semantic Hashing** (Salakhutdinov & Hinton, 2009): ML-based binary codes for similar documents. "Reconstruction" means vector recovery, not semantic knowledge regeneration.
- **Content-Addressable Storage (IPFS CIDs):** Self-describing layered prefix encoding. Closest structural analogy to RKH, but no epistemological metadata and no regenerability.
- **Caveman Compression / ByteRover (2025-26):** LLM context compression. Related spirit (compress-then-reconstruct) but no structured hash, no epistemological typing.
- **Knowledge Card (ICLR 2024):** Plug-in LMs that generate background knowledge. Shares the "regenerate knowledge" idea but requires full model weights, not a compact identifier.

11.4 RAG Safety and Filtering

- **RAGAS** (2023): Post-generation evaluation metrics (faithfulness, answer relevance, context precision). Complementary to KFP's pre-retrieval filtering — RAGAS catches hallucinations after the fact, KFP prevents unsafe input.
- **CRAG** (Corrective RAG, 2024): Evaluates retrieval quality and triggers web search for low-confidence results. Operates on relevance, not evidence level. Does not classify epistemological quality.

- **Self-RAG** (2024): LLM self-reflects on retrieval quality. Runtime cost (multiple LLM calls per query). KFP's rule-based filtering has zero LLM cost.

11.5 Patent Landscape

8 EPO OPS queries + 7-system prior art analysis found no blocking prior art for either the base fingerprint or RKH. The nearest reference (IBM US2019251452A1, Evidence Grading) covers a single dimension without composition algebra or reconstruction.

12. Limitations

1. **Rule-based extraction.** 300+ patterns cover common cases. Domain-specific jargon may misclassify. Topic key extraction achieves 0.8-1.0 confidence only for domains with dictionary coverage; 35% of real-world articles fall to 0.2 (word fallback).
2. **Template reconstruction quality.** 76% mean dimension fidelity without LLM (90% per-dimension, 33% full-match). LLM-assisted reconstruction achieves 82% evidence accuracy but has not been benchmarked for full reconstruction fidelity yet.
3. **Single-author validation.** The 82% inconsistency finding, 191-article ingest, and RAG benchmark are from internal datasets. Independent replication is needed.
4. **No external consumers.** The capture bridge and TRE integration are internal. No external system uses KFP in production.
5. **Agent identity is early.** Based on a single 4-agent experiment.
6. **Knowledge DNA is lossy.** Regenerated articles are semantically equivalent but not bit-identical to originals. Topic keys with low confidence produce poor reconstructions.
7. **Topic key format is limited.** Maximum 3 hierarchical levels (entity.aspect.qualifier) cannot represent subject-object pairs. A v2 format may be needed.
8. **RAG benchmark is simulated.** The 500-annotation ground truth and 22-chunk medical RAG test do not use a real vector store or embedding model. Production RAG

results may differ. The PubMedQA-inspired benchmark uses constructed sentences, not original PubMedQA data. Rule-based filtering achieves 86% reduction, not 100% — borderline cases leak through.

9. **Extraction bottleneck.** The primary quality bottleneck is first extraction (48% Knowledge Type accuracy on diverse texts), not reconstruction. Improving the extractor would improve all downstream results.

13. Conclusion

KFP demonstrates that epistemological identity — describing *what kind* of knowledge something is, not just *how good* — is a tractable and useful problem. The composition algebra makes quality degradation visible and irreversible. Dual-fingerprinting catches metadata lies. Agent identity reveals team blind spots invisible to consensus mechanisms. Epistemological RAG filtering reduces speculative knowledge entering LLM context by 86% — the first pre-retrieval safety layer based on evidence quality rather than semantic similarity.

RKH extends this identity into an address: a compact string from which the knowledge can be semantically reconstructed. Knowledge DNA compresses an entire knowledge base into one line per article — 13x smaller than JSON, human-readable, git-diffable, and self-healing through regeneration. The 191-article ingest demonstrates that the full pipeline (extraction, fingerprinting, RKH, wiki, DNA, regeneration) works on real data at sub-second speed.

The system is fully deterministic (no LLM required for extraction or template reconstruction), implemented in standard-library Python (33 modules, 914 tests, 8,875 lines of code, 7 MCP tools, 4 integration bridges + 1 specification), installable via `pip install kfp-fingerprint`, and is the first framework to combine epistemological typing, closed composition algebra, cryptographic verification, semantic reconstruction, and evidence-gated retrieval. An interactive demo (`python3 -m kfp demo`) demonstrates the full pipeline in 30 seconds.

The primary open question is adoption. The initial results (82% misclassification detection, 76% reconstruction fidelity, 191-article DNA round-trip, 86% unsafe chunk reduction in RAG

filtering against 500 ground truth annotations, 66-82% evidence classification accuracy on PubMedQA-inspired benchmark) suggest practical value, but external validation and production deployment are needed. Knowledge DNA's potential as a cross-system exchange format — analogous to SBOM for software — requires multiple independent implementations to establish.

A Note on Process and Transparency

This project was developed with AI coding assistance. Architecture, research questions, and interpretation of results are human contributions. The author takes full responsibility for all claims, proofs, and conclusions presented.

References

1. Salakhutdinov, R. & Hinton, G. (2009). Semantic Hashing. *International Journal of Approximate Reasoning*, 50(7), 969-978.
2. GRADE Working Group (2004). Grading quality of evidence and strength of recommendations. *BMJ*, 328(7454), 1490.
3. Moreau, L. & Missier, P. (2013). PROV-DM: The PROV Data Model. *W3C Recommendation*.
4. Hintikka, J. (1962). Knowledge and Belief. *Cornell University Press*.
5. IBM (2019). US2019251452A1: Grading Sources and Managing Evidence.
6. Bosch (2022). US2022284309A1: Aligning Knowledge Graphs Using Subgraph Typing.
7. Benet, J. (2014). IPFS - Content Addressed, Versioned, P2P File System. *arXiv:1407.3561*.
8. Wang, Y. et al. (2024). Knowledge Card: Filling LLMs' Knowledge Gaps with Plug-in Specialized Language Models. *ICLR 2024*.
9. NTIA (2021). The Minimum Elements For a Software Bill of Materials (SBOM). *U.S. Department of Commerce*.
10. Es, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv:2309.15217*.
11. Yan, S. et al. (2024). Corrective Retrieval Augmented Generation. *arXiv:2401.15884*.

12. Asai, A. et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR 2024*.